I've already mentioned third-party identity providers such as Auth0 and Azure Active Directory. These Software as a Service (SaaS) solutions take care of the user-management and authentication aspects of your app for you. If you're in the process of moving apps to the cloud generally, solutions like these can make a lot of sense.

If you can't or don't want to use these third-party solutions, I recommend you consider using the ASP.NET Core Identity system to store and manage user details in your database. ASP.NET Core Identity takes care of most of the boilerplate associated with authentication, but it remains flexible and lets you control the login process for users if you need to.

NOTE ASP.NET *Core* Identity is an evolution of the legacy .NET Framework ASP.NET Identity system, with some design improvements and update to work with ASP.NET Core.

By default, ASP.NET Core Identity uses EF Core to store user details in the database. If you're already using EF Core in your project, this is a perfect fit. Alternatively, it's possible to write your own stores for loading and saving user details in another way.

Identity takes care of the low-level parts of user management, as shown in table 23.1. As you can see from this list, Identity gives you a lot, but not everything—by a long shot!

Table 23.1 Which services are and aren't handled by ASP.NET Core IdentityCH23.fm